Selective State Space Models: Exercise Solutions

NeurIPS 2026 Education Track — Instructor Copy

Solution 1 — Discretization by hand

(a) For $t \in [k\Delta, (k+1)\Delta)$, $x(t) = x_k$ is constant, so the ODE $h'(t) = ah(t) + bx_k$ is a linear first-order equation with constant forcing. Its solution starting from $h(k\Delta) = h_k$ is

$$h(t) = e^{a(t-k\Delta)}h_k + \frac{bx_k}{a}\left(e^{a(t-k\Delta)} - 1\right).$$

Evaluating at $t = (k+1)\Delta$ (i.e. setting $t - k\Delta = \Delta$) gives

$$h_{k+1} = \underbrace{e^{a\Delta}}_{\bar{a}} h_k + \underbrace{\frac{e^{a\Delta} - 1}{a} b}_{\bar{b}} x_k,$$

which is exactly $\bar{a} = e^{a\Delta}$, $\bar{b} = \frac{\bar{a}-1}{a}b$.

(b) With $a = -4$, $b = 2$, $\Delta = 0.25$: $\bar{a} = e^{-1} = 0.3679$, $\bar{b} = \frac{0.3679-1}{-4} \cdot 2 = 0.3161$.

(c) With $\Delta = 2.0$: $\bar{a} = e^{-8} = 0.0003$, $\bar{b} = 0.4998$. The system is still stable ($|\bar{a}| < 1$) because $a < 0$ for *any* $\Delta > 0$ — $\bar{a} = e^{a\Delta}$ is a strictly decreasing positive function of Δ when $a < 0$, so it can never exceed 1. A larger Δ only makes the system decay *faster* (more of the previous state is forgotten each discrete step); it cannot make a system with $a < 0$ unstable. (Note: this is specific to first-order ZOH of a genuinely stable continuous system — it is not a general statement about discretizing ODEs.)

(d) Matches: `discretize_zoh(-4, 2, 0.25)` returns `(0.3679, 0.3161)`.

Solution 2 — Predicting tensor shapes

With $B = 16$, $L = 30$, `in_dim`$= 4$, `state_dim`$= 12$:

(a) `A`: shape `(12,)` — one decay rate per state channel, independent of batch/time.

(b) `Bt`: shape `(16, 30, 12)` — `nn.Linear(4, 12)` applied to the last dimension of `x` with shape `(16, 30, 4)`.

(c) `Ct`: shape `(16, 30, 12)`, same reasoning as `Bt`.

(d) `delta`: shape `(16, 30, 1)` — `nn.Linear(4, 1)` then softplus, which is shape-preserving.

(e) At a fixed `t`: `delta[:, t]` has shape `(16, 1)`; multiplying by `A` with shape `(12,)` broadcasts to `(16, 12)` — this is `a_bar`. `b_bar = delta[:, t] * Bt[:, t]` is `(16, 1)` times `(16, 12)`, broadcasting to `(16, 12)`.

(f) `h`: `(16, 12)` both before the loop (`torch.zeros(B, state_dim)`) and at every step inside it — the recurrence never changes its shape, only its values.

(g) Each per-step `y = (Ct[:, t] * h).sum(-1, keepdim=True)` is `(16, 12)` summed over the last axis with `keepdim=True`, giving `(16, 1)`. Stacking 30 of these along `dim=1` gives the final shape `(16, 30, 1)`. (Aside: `self.out_proj` is defined in `__init__` but `out_proj_identity` does not actually apply it — it is an unused hook left for extensions, not a bug in the shape logic.)

Solution 3 — Breaking selective copying

(a) Track `running_sum` and `running_count`, updated at each marker, and set `target[b,t,0] = running_sum/running_count` (or 0 before the first marker).

(b) *Prediction: only approximately, not exactly.* A single-channel state update $h_t = \bar{a}_t h_{t-1} + \bar{b}_t$ is an affine (scale-and-add) operation. An exact running average requires the *new* value to be blended in with weight $\frac{1}{\text{count}_t}$ and the *old* average shrunk by weight $\frac{\text{count}_t - 1}{\text{count}_t}$ — i.e. the mixing coefficient must equal a specific, exactly-changing fraction tied to how many markers have been seen. Since $\bar{a}_t$ is produced by a learned continuous function of x_t (via Δ_t), the network can only *approximate* this schedule from the training distribution; it has no built-in counter and no guarantee of representing $1/\text{count}_t$ exactly for counts beyond what it saw in training.

(c) Empirically, both models' losses are non-zero (typically higher than on the original task, since the target now changes more gradually and errors compound), and the selective model still outperforms the fixed model — but the selective model's loss does not go to zero at the precision an exact-average computation would achieve, consistent with (b).

(d) The minimal change is to give the model an explicit way to compute (or be told) the marker count, and use it directly as the state update coefficient — for example, a dedicated counting channel that increments by exactly 1 at each marker (itself learnable, or hardcoded) and feeds $1/\text{count}_t$ into Δ_t for the value channel. This separates "count how many markers" (a task an SSM state can represent exactly, since incrementing by a fixed amount is a trivial fixed-coefficient recurrence) from "average the values" (which requires that exact count as an input, not an approximation of it).

Solution 4 — Scan depth and padding

(a) $L = 100 \Rightarrow n = 101$. The smallest power of two ≥ 101 is $2^7 = 128$, so `padded_n` $= 128$.

(b) Rounds $= \lceil \log_2 n \rceil$. For $n = 101$: $\log_2 101 \approx 6.658$, so $\lceil \cdot \rceil = 7$ rounds. (Equivalently: `step` takes values $1, 2, 4, 8, 16, 32, 64$ before reaching 128 — 7 iterations of the `while` loop.)

(c) $L = 1000 \Rightarrow n = 1001 \Rightarrow$ rounds $= \lceil \log_2 1001 \rceil = 10$. $L = 1{,}000{,}000 \Rightarrow n = 1{,}000{,}001 \Rightarrow$ rounds $= \lceil \log_2 1{,}000{,}001 \rceil = 20$. Going from $L = 10^3$ to $L = 10^6$ — a $1000\times$ increase in sequence length — only doubles the round count ($10 \to 20$), because rounds grow as $\log_2 L$ while L grows linearly. This is exactly why the parallel scan's advantage widens dramatically at long sequence lengths (matching the growing speedup trend measured in Notebook 3).

(d) Rounds is a step function of L (it only increases at powers of two), while $0.01L$ grows continuously. Computing both: rounds stays at 10 for every L with $n = L + 1 \in (512, 1024]$, i.e.

$L \in [512, 1023]$. Within that range, $0.01L$ passes 10 exactly at $L = 1000$ — but at $L = 1000$, rounds $= 10$ and $0.01L = 10.00$ exactly, which does *not* satisfy the strict inequality rounds $< 0.01L$. The first L where it is strictly satisfied is $\boxed{L = 1001}$ (rounds $= 10$, $0.01L = 10.01$).

Solution 5 — SSM vs. attention: compute and memory

(a) Self-attention: computing the full $L \times L$ score matrix and applying it costs $O(L^2 d)$ FLOPs. Selective SSM via parallel scan: each of the $O(\log L)$ rounds does $O(L)$ combine operations, each over an $O(N)$-sized (or $O(Nd)$ if applied per-channel) state, giving $O(LNd \log L)$ total — commonly simplified to $O(LNd)$ since the $\log L$ factor is small in practice and the dominant, quoted cost is the linear-in-L term.

(b) Attention at inference must retain the KV cache: keys and values for all L tokens seen so far, costing $O(Ld)$ memory (specifically $2Ld$ scalars for K and V combined, ignoring the head-splitting constant). A selective SSM only needs to retain its fixed-size hidden state h_t, costing $O(Nd)$ memory (one length-N state per channel, d channels) — independent of L.

(c) Setting SSM memory equal to attention memory: $Nd = 2Ld \Rightarrow L^* = \dfrac{N}{2}$. Notice d cancels — the crossover point depends only on the state size N, not on model width. For $N = 64$: $L^* = 32$. In words: once the context exceeds just 32 tokens, the SSM's fixed-size state is already cheaper per decoding step than attention's growing KV cache — a smaller crossover point than intuition might suggest, precisely because the KV cache grows without bound while the SSM state does not.

(d) This analysis overstates the practical advantage for tasks requiring *exact, arbitrary-position retrieval* — e.g. precisely copying or looking up a specific token from far back in the context. As discussed on Slide 15 (citing Jelassi et al., 2024), a state of fixed size N can only compress so much history, so on tasks demanding lossless recall of arbitrary past tokens, attention's explicit per-token KV cache is a genuine advantage that this memory-only comparison does not capture — the two architectures are not interchangeable purely because one uses less memory per step.